

Hệ Thống Đấu Giá Trực Tuyến (Online Auction Platform)

THÔNG TIN NHÓM THỰC HIỆN ĐỒ ÁN

- **Ngành:** Công nghệ Thông tin - Trường Đại học Công nghệ (UET)
 - **Môn học:** Lập trình nâng cao
 - **Tên đồ án:** Xây dựng Hệ thống Đấu giá Trực tuyến thời gian thực (Real-time)
 - **Danh sách thành viên:**
 1. **Lê Đức Tiến** – MSSV: 25021981
 2. **Ngô Trung Kiên** – MSSV: 25021818
 3. **Lã Quốc Sử** – MSSV: 25020354
 4. **Hoàng Khánh Huy** – MSSV: 25021796
-

PHẦN 1: GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1. Mục tiêu đồ án

Xây dựng một nền tảng đấu giá tài sản trực tuyến hoàn chỉnh, mô phỏng các nền tảng đấu giá thực tế. Hệ thống được thiết kế tối ưu để giải quyết ba bài toán cốt lõi trong lập trình hướng đối tượng OOP:

- Đảm bảo tính đồng bộ dữ liệu thời gian thực (Real-time) giữa nhiều người dùng.
- Chống tắc nghẽn, xung đột cơ sở dữ liệu khi có lượng truy cập đọc/ghi đồng thời lớn.
- Bảo vệ an toàn dữ liệu cá nhân và tính công bằng trong quá trình trả giá.

2. Phạm vi thực hiện

- **Mô hình triển khai:** Kiến trúc Client - Server đa luồng.
 - **Phân quyền hệ thống:** Hỗ trợ 3 vai trò độc lập Admin – Seller – Bidder.
 - **Luồng nghiệp vụ chính:** Quản lý tài khoản, Nạp/duyet tiền ảo, Đăng tải sản phẩm, Trả giá trực tiếp (Manual-bid), Đặt giá tự động (Auto-bid) và thanh toán tự động khi phiên đấu giá kết thúc.
-

PHẦN 2: KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

Hệ thống được thiết kế theo mô hình **Client-Server đa luồng (Multi-threaded)**, phân tách rõ ràng trách nhiệm của từng tầng nhằm đảm bảo nguyên tắc Clean Code và dễ dàng mở rộng:

[TẦNG CLIENT (JavaFX - Mô hình MVC)]

[Giao diện tĩnh (views)]

- login.fxml, register.fxml
- Lobby.fxml, Lobbypane.fxml



[Lớp điều khiển (action)]

- AuthenticationController
- MainUIController
- SellingJobsController



[Xử lý Mạng (Utilities)]

- SocketClient
- SocketListener (Background)

[TẦNG SERVER (Đa luồng và phân lớp)]

[Lắng nghe mạng (network)]

- AuctionServer
- ClientHandler (ThreadPool)



[Bộ định tuyến (command)]

- CommandDispatcher
- LoginCommand, RegisterCommand
- BidCommand, AddAuctionCommand



[Xử lý nghiệp vụ (core)]

- AuthService, AuctionService
- Logging (SLF4J / Logback)



[Truy xuất Dữ liệu (repository)]

- DatabaseManager (Singleton)
- BaseDAO (Lớp cơ sở)
- UserDao, ItemDAO, AuctionDAO

[GIAO TIẾP MẠNG]

- TCP/IP Sockets
- Gson: Java Object ↔ JSON String
- Dữ liệu: payload

[CƠ SỞ DỮ LIỆU (SQLite)]

Tệp tin: auction.db (SQLite)
Chế độ: WAL Mode Enabled

1. Tầng Client (Giao diện hiển thị người dùng)

Hệ thống vận hành theo kiến trúc MVC chặt chẽ. Toàn bộ file giao diện được định nghĩa tĩnh qua XML (nhóm `views`). Logic điều khiển sự kiện và kiểm tra tính hợp lệ dữ liệu đầu vào được xử lý tại các Controller đích danh (`AuthenticationController`, `MainUIController`, `SellingJobsController`). Luồng mạng được cô lập hoàn toàn tại `SocketListener` chạy trên một Background Thread riêng biệt, đảm bảo FPS của giao diện JavaFX không bị sụt giảm hay đóng băng khi chờ máy chủ xử lý.

2. Tầng Giao tiếp mạng (Network Protocol)

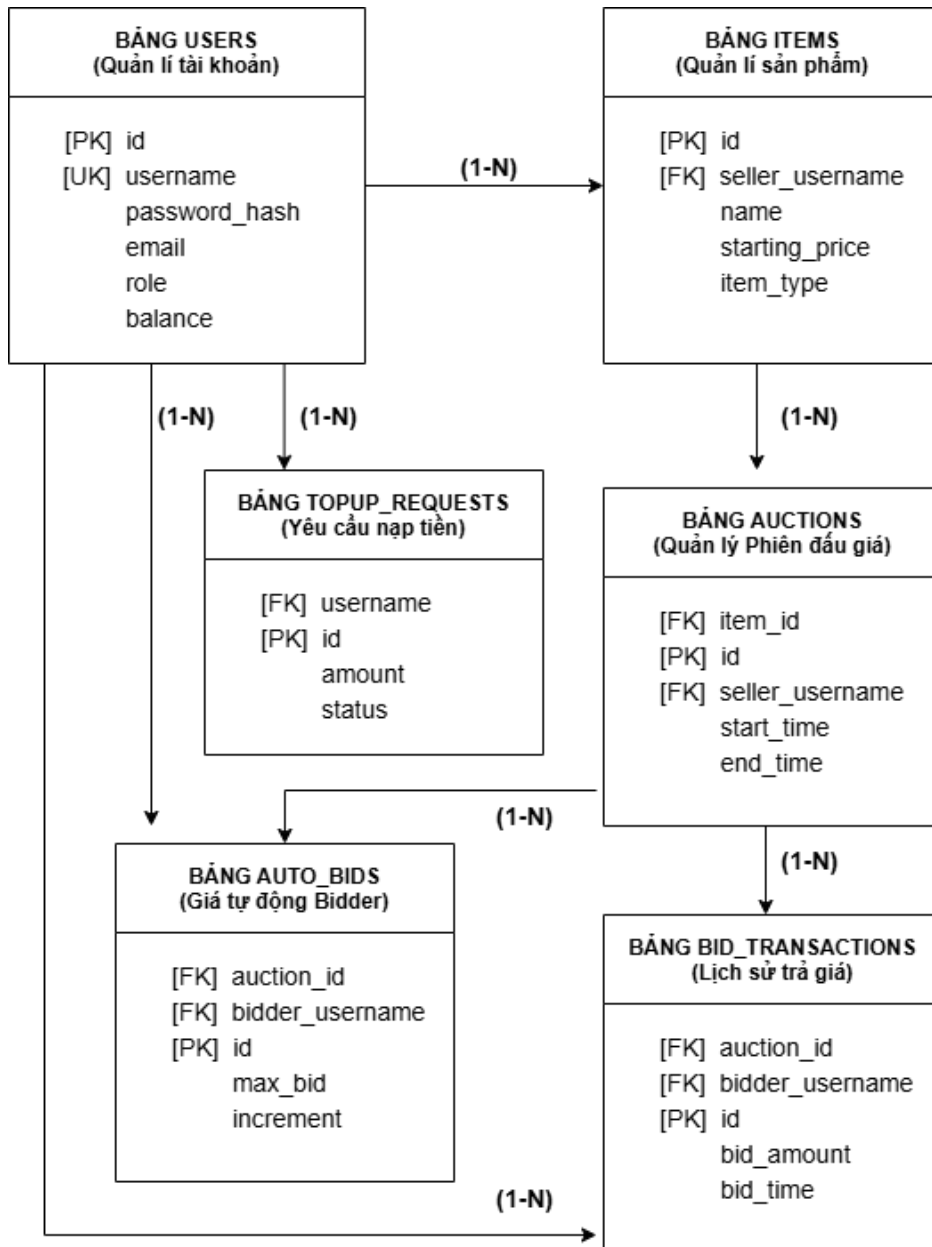
Kết nối hai chiều liên tục được duy trì qua TCP/Socket. Mọi thông tin trao đổi được cấu trúc hóa thông qua các Data Transfer Object (nằm trong package `payload`) và áp dụng cơ chế Tuần tự hóa (Serialization) thành định dạng văn bản JSON thông qua thư viện Gson. Giải pháp này xóa bỏ hoàn toàn rủi ro lỗi IndexOutOfBoundsException do cắt chuỗi String thô sơ.

3. Tầng Server (Xử lý nghiệp vụ phân lớp)

Máy chủ tiếp nhận kết nối đa luồng thông qua cơ chế `ThreadPool` tại lớp `ClientHandler`. Luồng tín hiệu mạng đi vào không tương tác trực tiếp với cơ sở dữ liệu. Thay vào đó, chúng bị đánh chặn tại `CommandDispatcher` (áp dụng mẫu thiết kế Command Pattern). Tùy thuộc vào mã lệnh (Action Code), gói tin sẽ được chuyển tiếp thành các đối tượng thi hành cụ thể trước khi bàn giao cho tầng nghiệp vụ lõi xử lý. Đồng thời, toàn bộ trạng thái hoạt động được giám sát chặt chẽ qua hệ thống Logging (SLF4J kết hợp Logback).

4. Tầng Dữ liệu (Repository & Database)

Mọi thao tác truy vấn cơ sở dữ liệu được quản lý bởi một điểm truy cập duy nhất `DatabaseManager` (Singleton Pattern). Các nghiệp vụ SQL được bóc tách rõ ràng vào các lớp DAO cụ thể (tất cả đều kế thừa phương thức chuẩn từ `BaseDAO`). Cơ sở dữ liệu SQLite được thiết lập ở chế độ WAL (Write-Ahead Logging) kết hợp cùng từ khóa `synchronized` tại các phương thức đặt giá, giải quyết triệt để xung đột cấp phát tài nguyên (Race Condition) khi Server có hàng chục truy vấn được kích hoạt cùng một thời điểm.



* Chú thích:

[PK]: Primary Key (Khóa chính)

[UK]: Unique Key (Khóa độc nhất - Dùng làm gốc tham chiếu)

[FK]: Foreign Key (Khóa ngoại - Trỏ về bảng khác)

(1-N): Mối quan hệ Một - Nhiều (Ví dụ: 1 User có Nhiều yêu cầu nạp tiền).

PHẦN 3: CÁC CHỨC NĂNG ĐẠT ĐƯỢC

Dưới đây là bảng phân tích chuyên sâu về các chức năng đã hoàn thiện, hướng giải quyết và lý do lựa chọn hướng đó :

STT	Chức năng	Hướng giải quyết & Kỹ thuật áp dụng	Lý do lựa chọn & Hiệu quả
1	Quản lý Tài khoản & Phân quyền (Đăng ký, Đăng nhập, Quản trị)	- Áp dụng tính Kế thừa (OOP): Lớp cha User phân nhánh thành Admin, Seller, Bidder. - Băm mật khẩu bằng thuật toán PBKDF2.	- OOP giúp dễ dàng phân luồng giao diện (UI) và tính năng cho từng Role - PBKDF2 chống lại các cuộc tấn công vét cạn (Brute-force) kể cả khi lộ file .db.
2	Quản lý Sản phẩm & Phiên đấu giá	- Sử dụng tính Đa hình: Lớp Item rẽ nhánh thành Art, Electronics... - Trạng thái phiên đấu giá được cập nhật tự động bằng Thread chạy ngầm phía Server.	- Dễ dàng mở rộng thêm các loại tài sản mới sau này mà không cần sửa core logic. - Đảm bảo vòng đời phiên đấu giá hoạt động trơn tru không cần admin can thiệp tay.
3	Đấu giá trực tiếp thời gian thực (Real-time Bidding)	- Sử dụng TCP/Socket thuần và mô hình Observer - Đóng gói gói tin dưới định dạng JSON bằng thư viện Gson. - Tách luồng mạng (SocketListener) khỏi luồng giao diện UI.	- Đảm bảo độ trễ thấp nhất. Việc truyền JSON loại bỏ hoàn toàn lỗi vỡ cấu trúc dữ liệu khi parse chuỗi - Giao diện Client mượt mà, không bị "đóng băng" khi chờ mạng phản hồi.
4	Xử lý đồng thời & Chống mất dữ liệu	- Sử dụng block synchronized tại AuctionService.	- Giải quyết bài toán Race Condition khi

STT	Chức năng	Hướng giải quyết & Kỹ thuật áp dụng	Lý do lựa chọn & Hiệu quả
	(Concurrency / Lost Update)	- Cấu hình SQLite sang chế độ WAL Mode (Write-Ahead Logging) + busy_timeout.	nhiều người cùng đặt giá 1 lúc. - WAL Mode loại bỏ triệt để lỗi khóa database (database is locked) trên môi trường Đa luồng.
5	Đặt giá tự động (Auto-bidding)	- Khởi tạo AutoBidThread quét liên tục. - Thuật toán tự động tính toán bước giá (Increment) để đưa người dùng lên Top 1 mà không vượt quá mức giá trần (MaxBid).	- Đảm bảo tính công bằng, hỗ trợ người mua không cần túc trực online liên tục để canh giá, đồng thời không tự ý đẩy giá lên quá cao vượt mức cần thiết.
6	Chống cướp hàng (Anti-sniping)	- Server thiết lập bộ đếm kiểm tra mốc thời gian: Tự động gia hạn thêm thời gian vào endTime nếu phát hiện có lệnh đặt giá hợp lệ ở những giây cuối cùng.	- Ngăn chặn triệt để hành vi dùng tool/bot đánh úp giành giật ở những giây cuối, đảm bảo môi trường đấu giá minh bạch.
7	Quản lý Ví điện tử ảo & Thanh toán	- Xây dựng cơ chế duyệt nạp tiền (Admin) và tự động trừ tiền trong Ví thông qua SQL Transaction. Sử dụng dữ liệu kiểu BIGINT cho tiền tệ.	- Đảm bảo tính toàn vẹn dữ liệu (ACID) cho số dư tài khoản ảo. Tránh sai sót thập phân khi thao tác với tiền tệ.
8	Đóng gói & Triển khai hệ thống	- Cấu hình maven-shade-plugin kết hợp class môi (launcher.java) để gộp toàn bộ JavaFX và thư viện bên thứ 3.	- Xuất bản thành file Executable Fat JAR độc lập. Giảng viên chỉ cần gõ 1 lệnh là chạy thẳng, không cần cài đặt môi trường phức tạp.